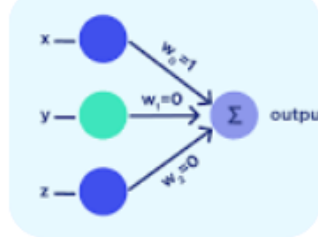


Perceptron and Perceptron Training

Dr. Sifat Momen

A perceptron is a fundamental building block of neural networks, essentially a simplified model of a biological neuron. It's a supervised learning algorithm that performs binary classification by learning a decision boundary, allowing it to categorize inputs into one of two classes. [🔗](#)



Here's a breakdown: [🔗](#)

Inputs:

Perceptrons receive multiple input values, which can represent features of data points.

Weights:

Each input is associated with a weight, representing its importance in the decision-making process.

Weighted Sum:

The perceptron calculates a weighted sum of the inputs, essentially combining the weighted inputs.

Activation Function:

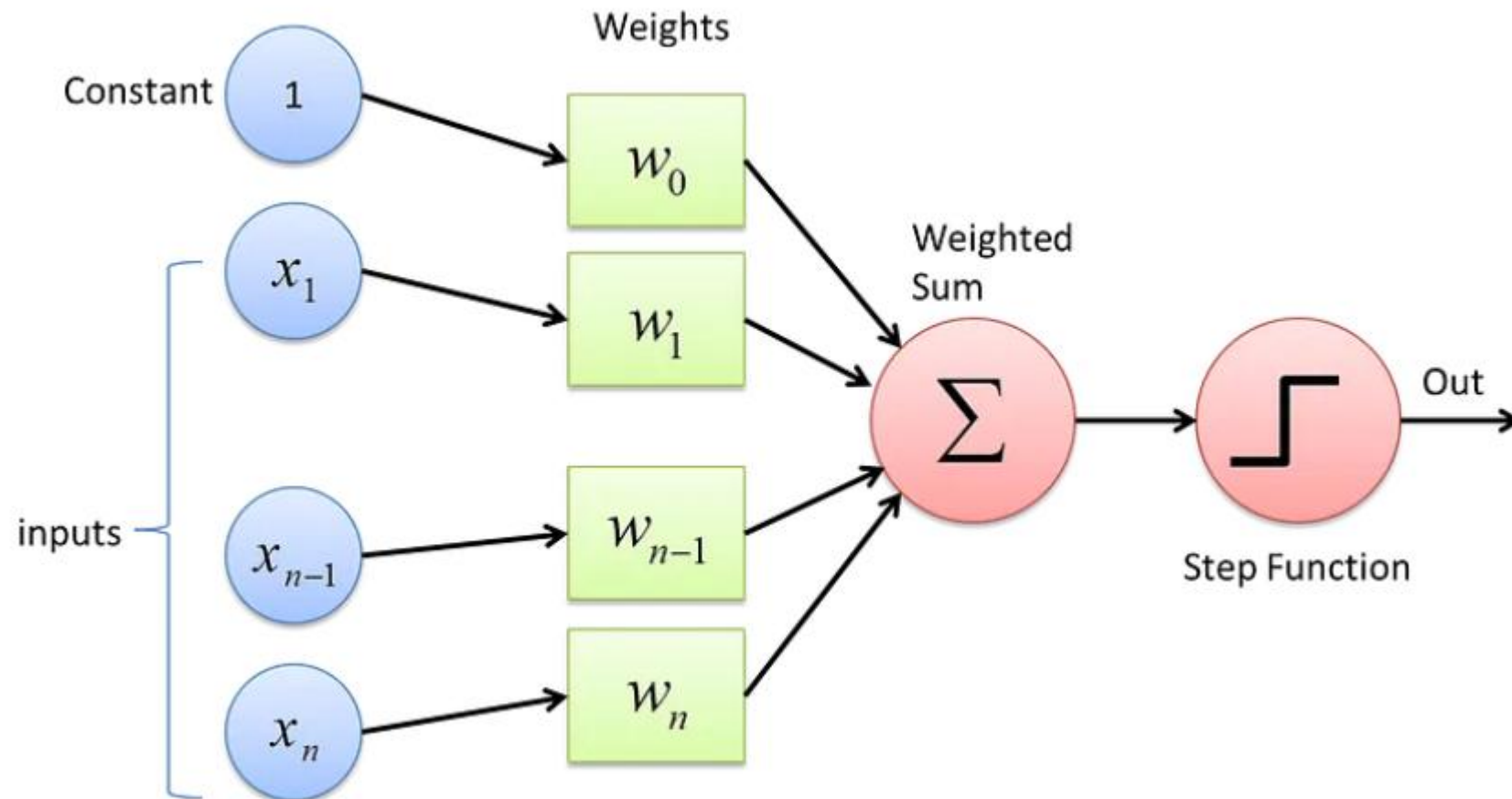
This sum is then passed through an activation function (like a step function) which determines the output of the perceptron.

Binary Output:

The output is typically a binary value (0 or 1), indicating which class the input belongs to.

In essence, a perceptron learns to adjust the weights of its inputs to correctly classify data, making it a core concept in the field of artificial neural networks. [🔗](#)

Linear Perceptron Architecture



Linear Perceptron

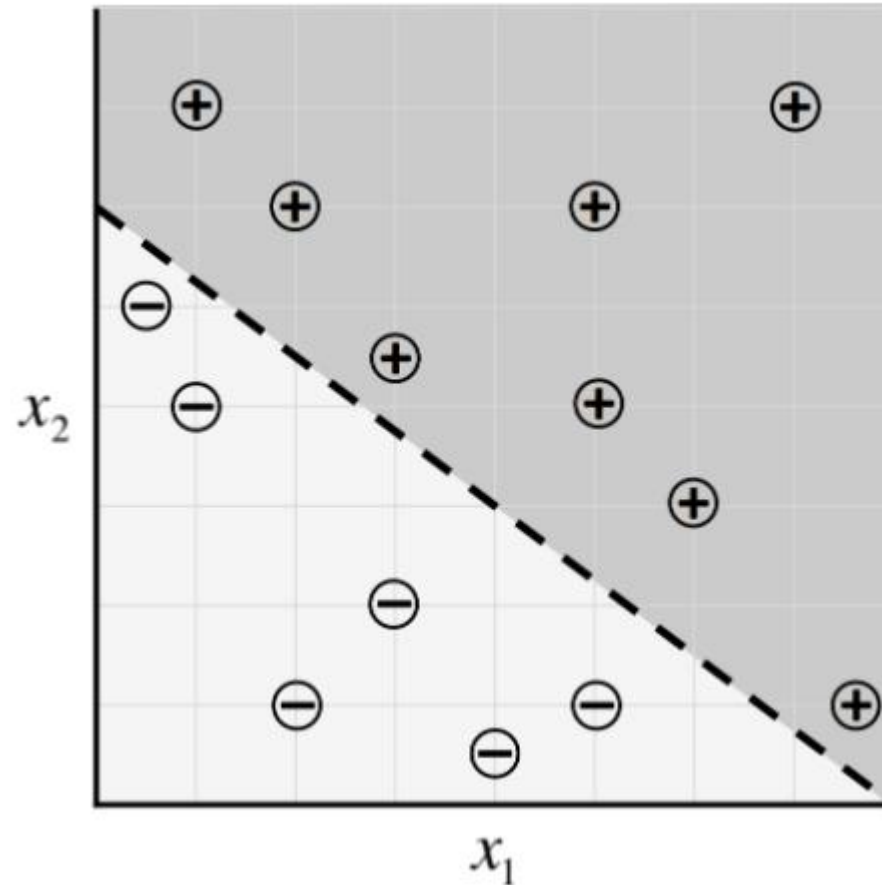
- Let's define our model to be $h(\mathbf{x}, \theta)$, for example,
 - $\mathbf{x} = [x_1 \ x_2]^T$ is the input, and
 - x_1 is the number of hours of sleep
 - x_2 is the number of hours spent on studying the previous day of an exam
 - $\theta = [\theta_0 \ \theta_1 \ \theta_2]^T$ is the model parameter
- We want to predict the exam performance. If the mark obtained is above or equal to the class average, the model is going to output a +1; otherwise a -1.

$$h(\mathbf{x}, \theta) = \begin{cases} -1 & \text{if } \mathbf{x}^T \cdot \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + \theta_0 < 0 \\ 1 & \text{if } \mathbf{x}^T \cdot \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + \theta_0 \geq 0 \end{cases}$$

Linear Perceptron

$$\theta = [-24 \ 3 \ 4]^T$$

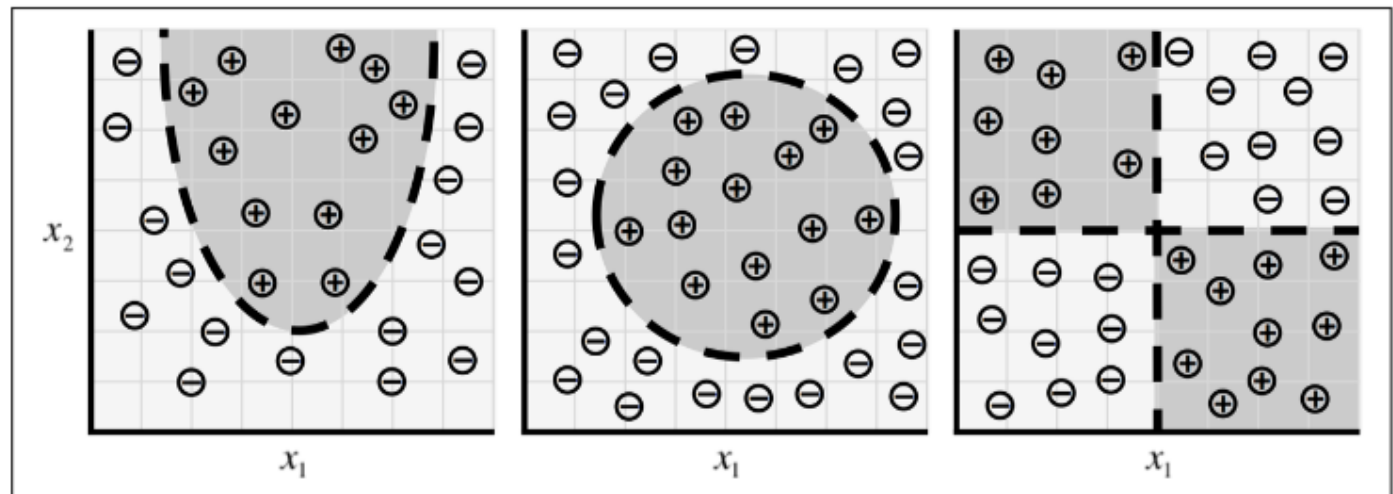
$$h(\mathbf{x}, \theta) = \begin{cases} -1 & \text{if } 3x_1 + 4x_2 - 24 < 0 \\ 1 & \text{if } 3x_1 + 4x_2 - 24 \geq 0 \end{cases}$$



2D -> line
3D -> plane
 ≥ 4 D -> hyperplane

Issues

- How do we know the right values of θ ?
- Linear perceptron model can learn provided that the dataset is linearly separable.
- As we move on to much more complex problems, such as object recognition and text analysis, our data becomes extremely high dimensional, and the relationships we want to capture becomes highly nonlinear.



Neuron

- Foundational unit of a human brain is a neuron.
- A tiny piece of a brain, about the size of grain of rice, contains over 10,000 neurons, each of which forms an average of 6,000 connections with other neurons.
- It's this massive biological network that enables us to experience the world around us.

Expressing linear perceptron as neurons

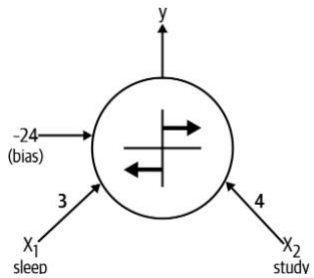
Artificial Neuron:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]$$

$$\mathbf{w} = [w_1, w_2, \dots, w_n]$$

Logits of a neuron = z

$$z = \sum_{i=1}^n w_i x_i + b$$



$$f(z) = \begin{cases} -1 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$h(\mathbf{x}, \theta) = \begin{cases} -1 & \text{if } 3x_1 + 4x_2 - 24 < 0 \\ 1 & \text{if } 3x_1 + 4x_2 - 24 \geq 0 \end{cases}$$

At its core, the neuron is optimized to receive information from other neurons, process this information in a unique way, and send its result to other cells. This process is summarized in [Figure 3-6](#). The neuron receives its inputs along antennae-like structures called *dendrites*. Each of these incoming connections is dynamically strengthened or weakened based on how often it is used (this is how we learn new concepts), and it's the strength of each connection that determines the contribution of the input to the neuron's output. After being weighted by the strength of their respective connections, the inputs are summed together in the *cell body*. This sum is then transformed into a new signal that's propagated along the cell's *axon* and sent off to other neurons.

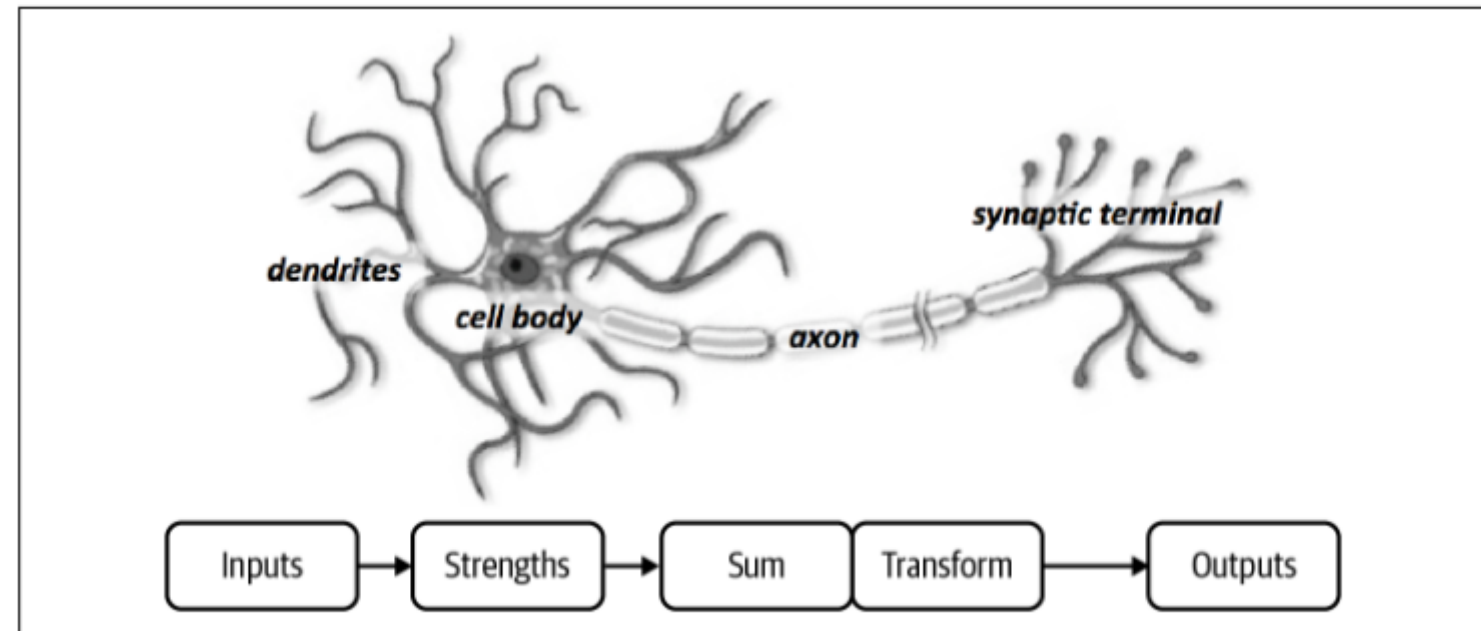


Figure 3-6. A functional description of a biological neuron's structure

Feed-forward neural networks

- A single neuron is not expressive enough to solve complicated learning problems.
- That's the reason, our brain is made of more than one neuron.
- In fact, the neurons in our brain are organized in layers.
- The human cerebral cortex (the structure responsible for most of the human intelligence) is made up of 6 layers.
- Information flows from one layer to another until sensory input is converted into conceptual understanding.

Feed-forward neural network

Input Layer:

- Introduces raw input data into the network
- Each neuron in this layer corresponds to one feature of the input data.
- The input layer does not perform any computation; it simply passes the data to the next layer.

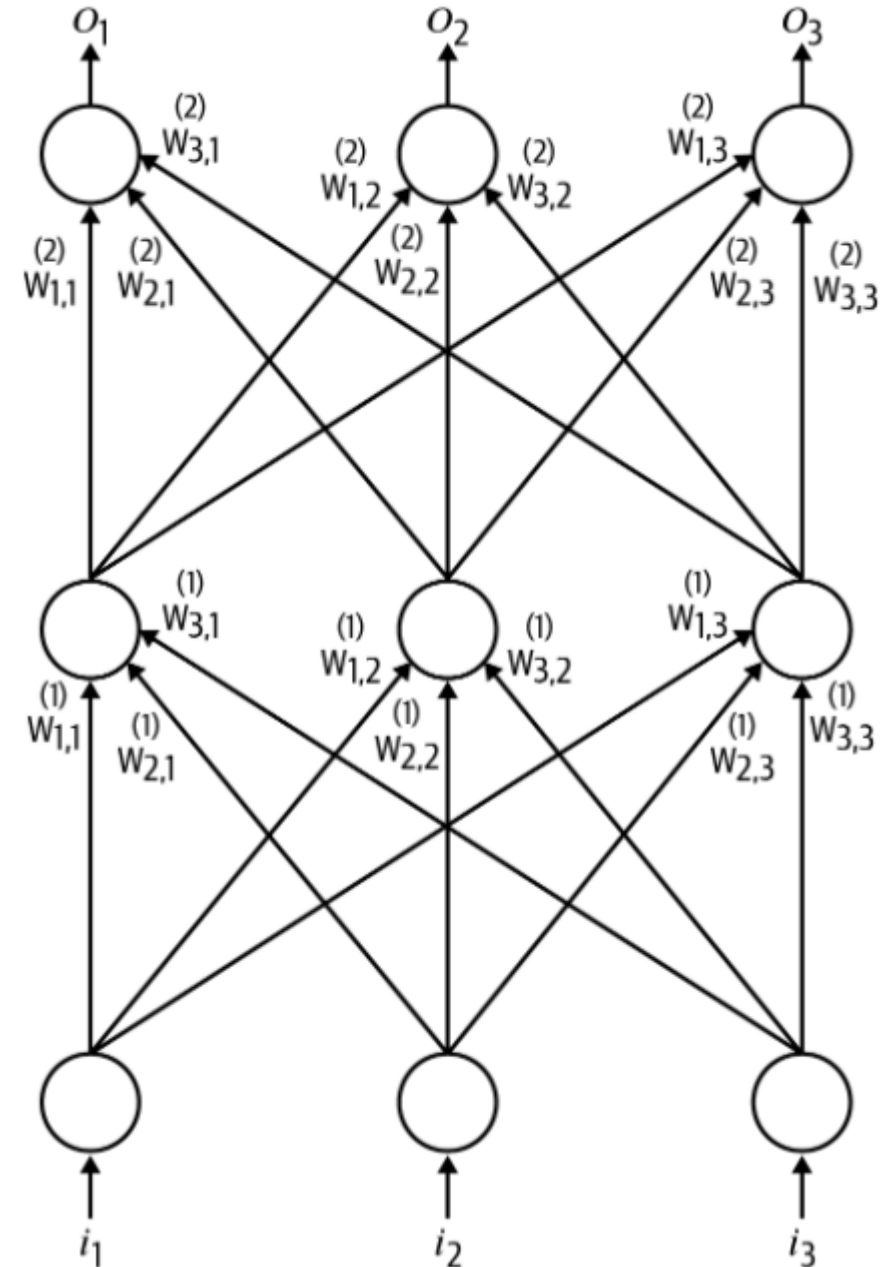
Hidden Layer:

- Hidden layers are where the majority of the learning happens.

Output Layer:

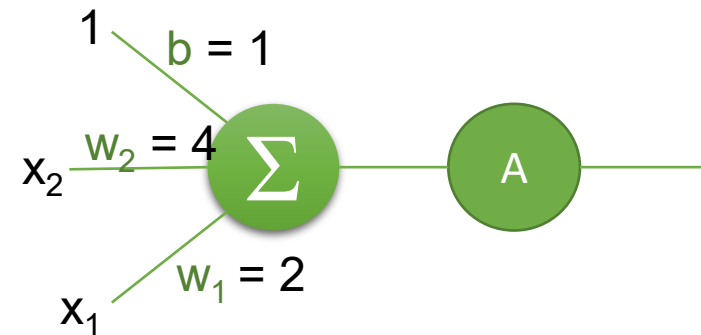
- Produces the final output of the network, which can be:
 - A single value (e.g., regression).
 - Class probabilities (e.g., classification).

Layer Type	Purpose	Common Activations
Input Layer	Pass raw input data	None
Hidden Layers	Learn features and patterns	ReLU, Tanh, Sigmoid
Output Layer	Generate predictions or classifications	Sigmoid, Softmax, Linear



Linear Neurons and their limitations

- Linear neurons are easy to compute with, but they run into serious limitations.
- Infact, it can be shown that any feed-forward neural network consisting of only linear neurons can be expressed as a network with no hidden layers.
- This is problematic because, as we discussed, hidden layers are what enable us to learn important features from the input data. In other words, to learn complex relationships, we need to use neurons that employ some sort of nonlinearity.



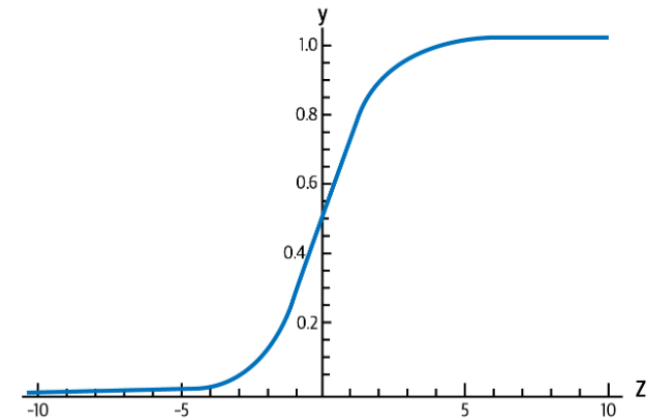
Sigmoid, Tanh, and ReLU Neurons

- They introduce nonlinearities in their computations.
- These neurons use different activation functions on the logits.
- Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Characteristics:

- **Range:** (0, 1)
- **Shape:** S-shaped curve.
- **Output Interpretation:** Often interpreted as a probability in classification problems.



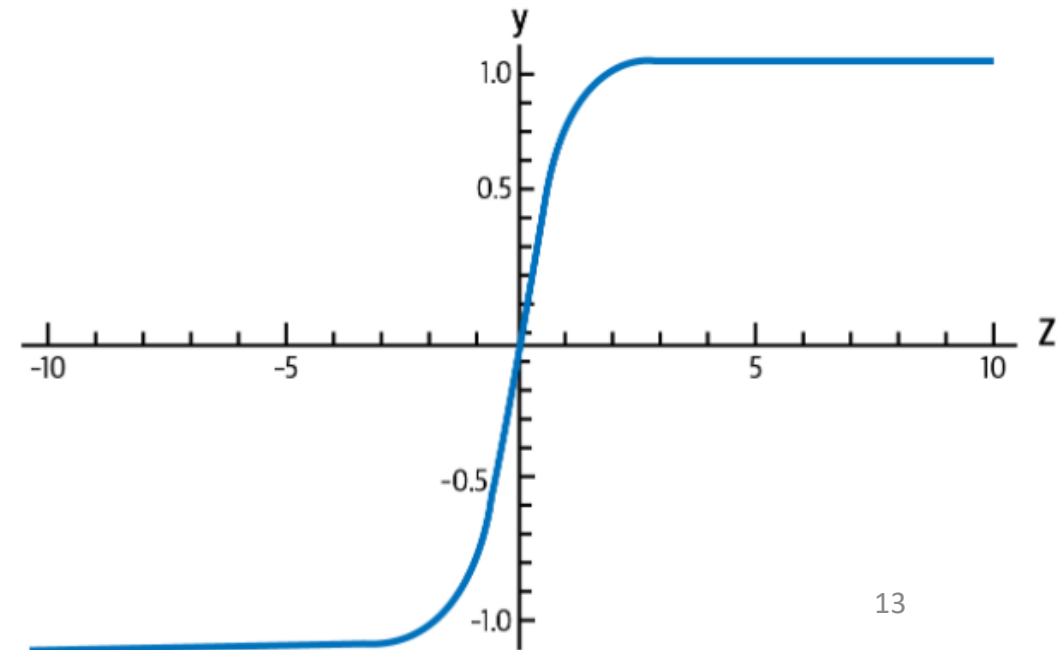
Tanh Neurons

- These neurons apply tanh (hyperbolic tangent) functions as activation functions

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Characteristics:

- **Range:** $(-1, 1)$
- **Shape:** S-shaped curve, similar to sigmoid but centered at 0.

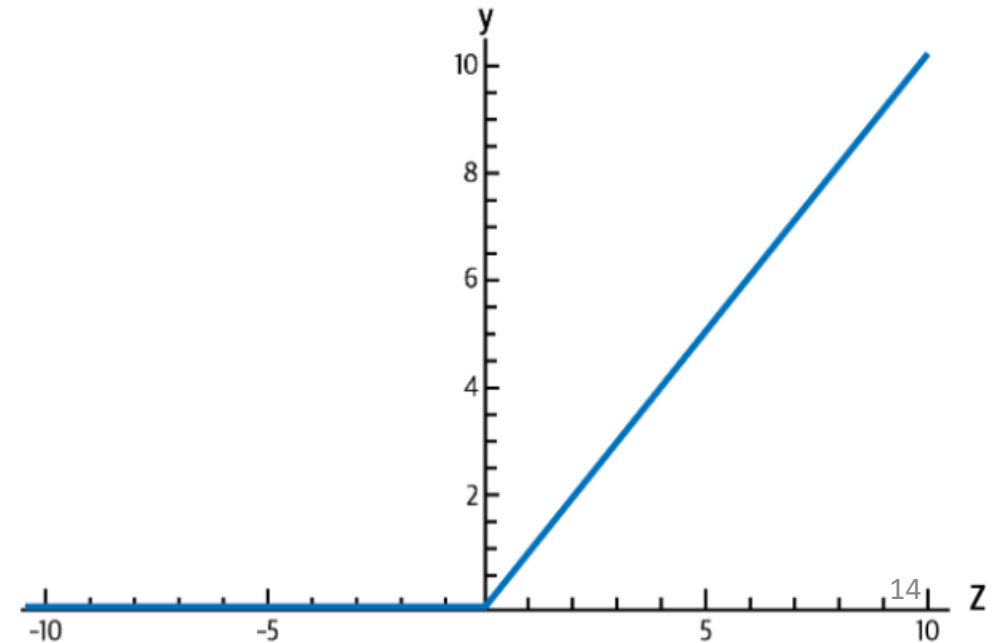


ReLU Neurons

- They apply ReLU functions as activation functions. $\text{ReLU}(z) = \max(0, z)$

Characteristics:

- Range: $[0, \infty)$
- Shape: Linear for positive inputs, flat (zero) for negative inputs.



Softmax Output layer

- Oftentimes, we want our output vector to be a probability distribution over a set of mutually exclusive labels.
- For example, let's say we want to build a neural network to recognize handwritten digits from the MNIST dataset. Each label (0 through 9) is mutually exclusive, but it's unlikely that we will be able to recognize digits with 100% confidence.
- Using a probability distribution gives us a better idea of how confident we are in our predictions.
- The desired output vector is of the following form, where $\sum_{i=0}^9 p_i = 1$:
- $[p_0 \ p_1 p_2 \ \dots \ p_9]$

$$y_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Comparison of Sigmoid Tanh, and ReLU

Feature	Sigmoid	Tanh	ReLU
Range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$
Zero-Centered	No	Yes	No
Vanishing Gradient	Yes	Yes	No (for $z > 0$)
Computational Cost	High	High	Low
Preferred For	Output layers	Hidden layers	Hidden layers

Example of Softmax function

Consider a classification task with three classes, and the logits are:

$$z = [2.0, 1.0, 0.1]$$

The softmax probabilities are:

$$P(y = 1) = \frac{e^{2.0}}{e^{2.0} + e^{1.0} + e^{0.1}} \approx 0.659$$

$$P(y = 2) = \frac{e^{1.0}}{e^{2.0} + e^{1.0} + e^{0.1}} \approx 0.242$$

$$P(y = 3) = \frac{e^{0.1}}{e^{2.0} + e^{1.0} + e^{0.1}} \approx 0.099$$

The predicted class is class 1, as it has the highest probability.

Role of Bias in a neural network

Each neuron in a neural network computes a weighted sum of its inputs, adds a bias term, and then applies an activation function:

$$a = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

Where:

- x_i : Input feature i .
- w_i : Weight corresponding to x_i .
- b : **Bias term**, a scalar that shifts the activation.
- f : Activation function (e.g., sigmoid, ReLU, etc.).
- a : Output of the neuron after activation.

Example of Bias in action

Without Bias:

Suppose you are fitting a line through the origin to separate two classes in a 2D space. The equation is:

$$y = w_1x_1 + w_2x_2$$

- The line always passes through the origin.
- This constraint might prevent the model from properly separating the classes if they are offset from the origin.

With Bias:

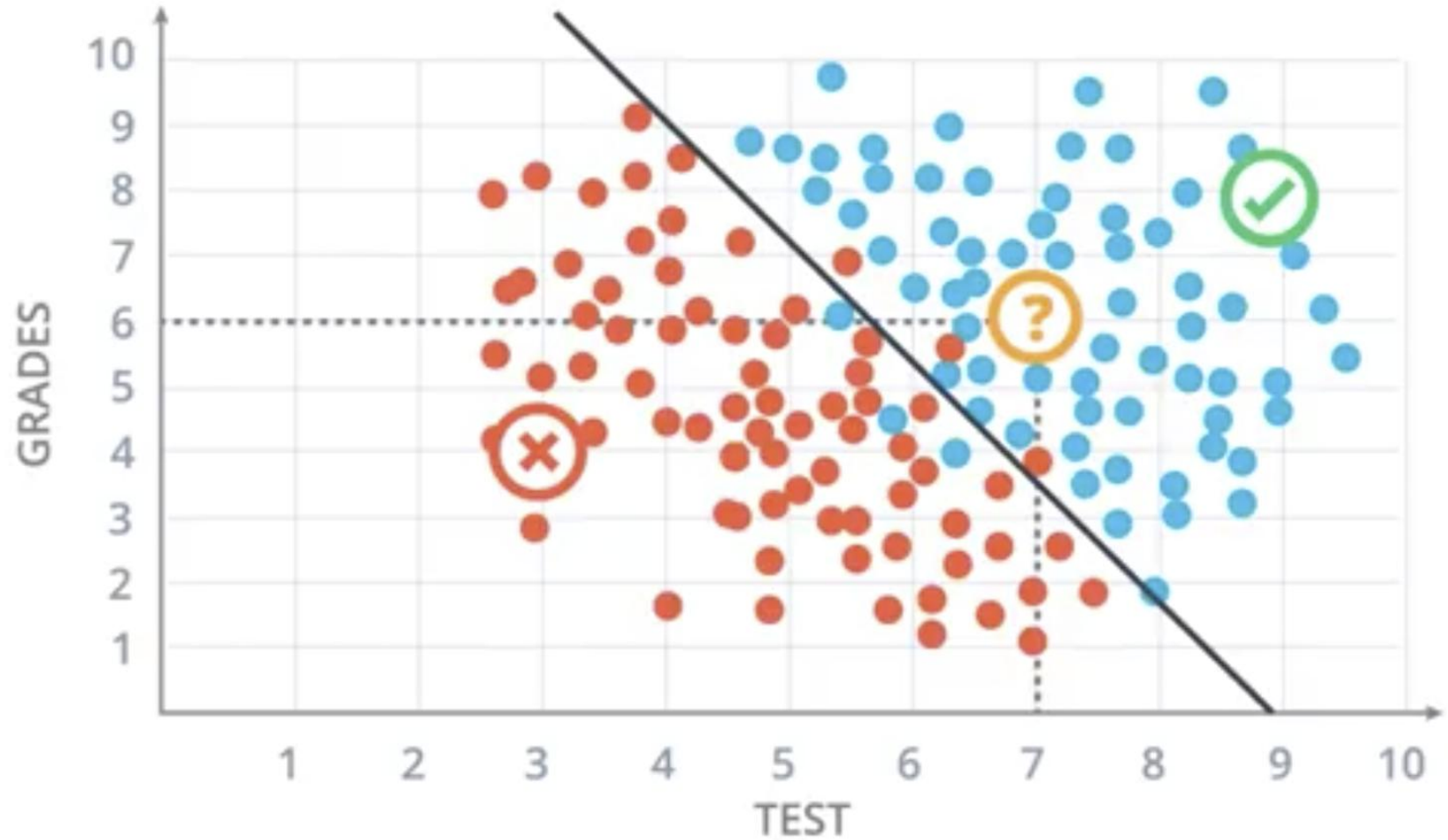
Now, with bias, the equation becomes:

$$y = w_1x_1 + w_2x_2 + b$$

- The line can shift up or down, providing more flexibility to fit the data.

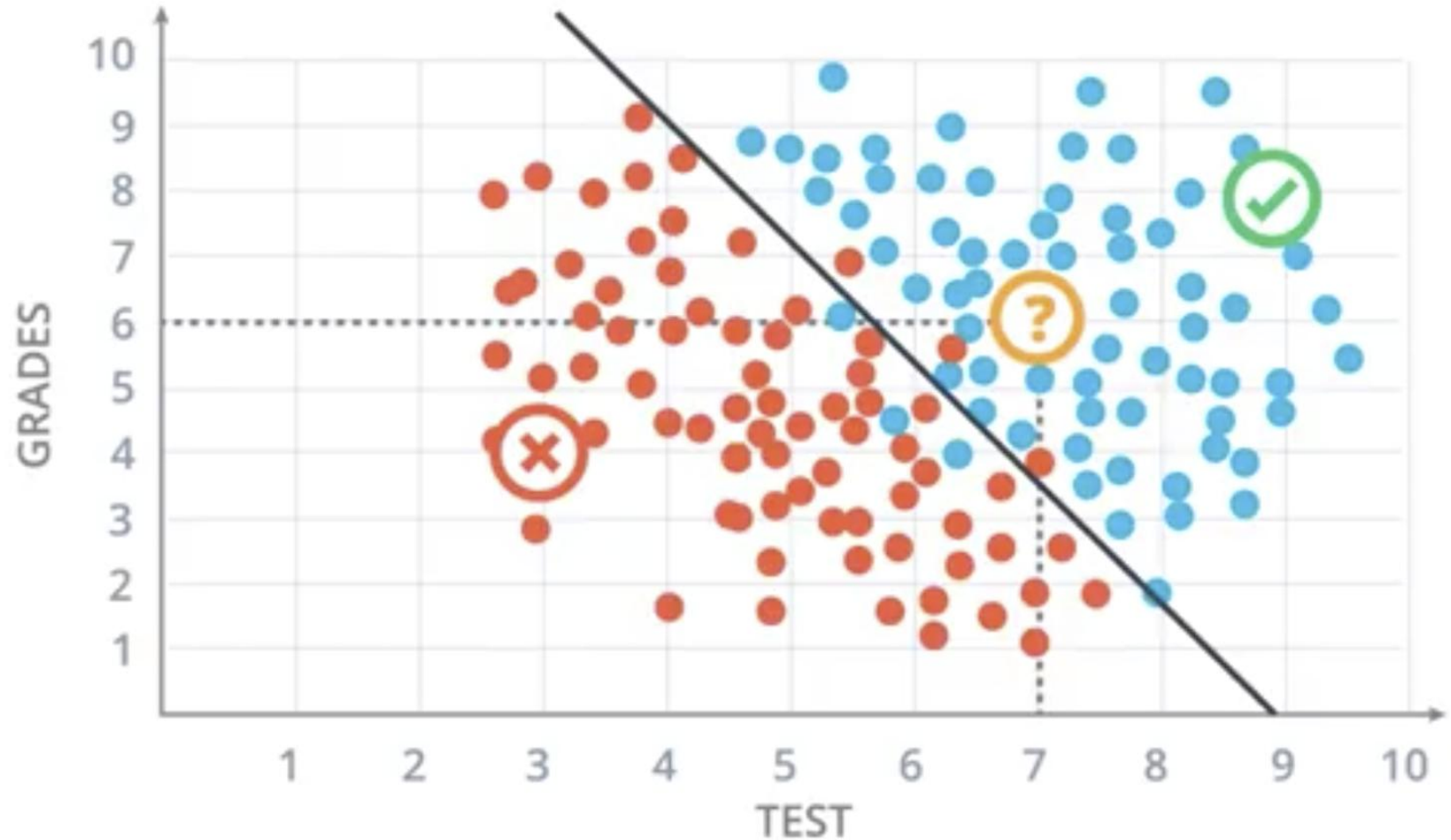
Decision boundary of Linear Perceptron

- $Ax + By + C = 0$



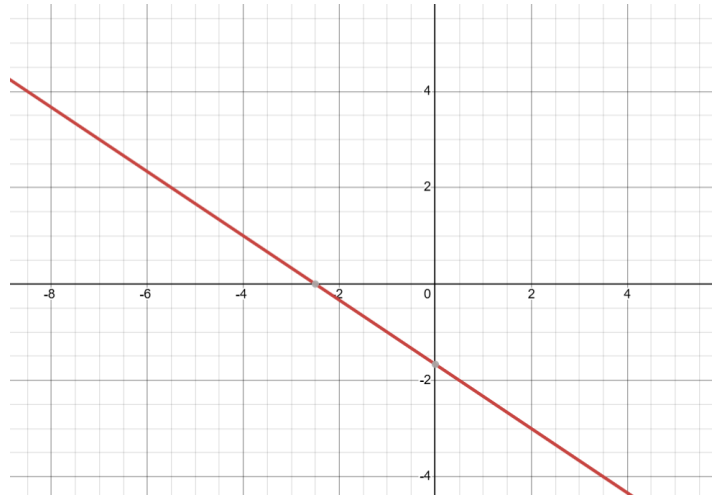
Perceptron Training – idea

- We start with a random line (by selecting random values of A, B, and C)
- We need to shift the decision boundary in such a way that it appropriately shifts the line

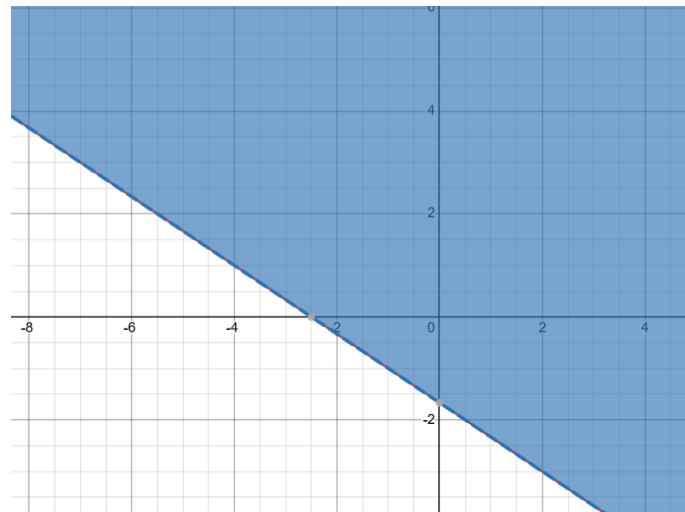


Perceptron Training – idea

- $2x + 3y + 5 = 0$

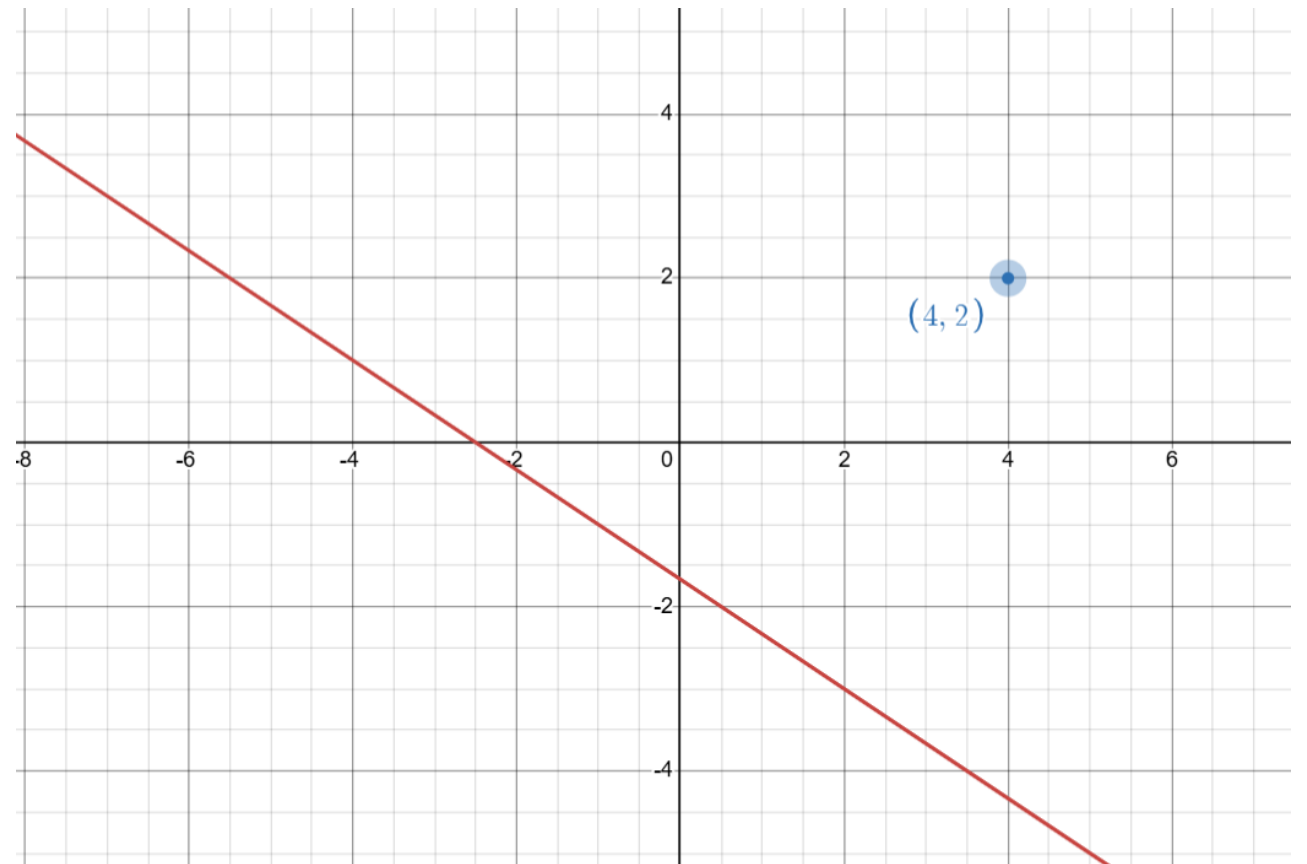


$$2x + 3y + 5 > 0$$



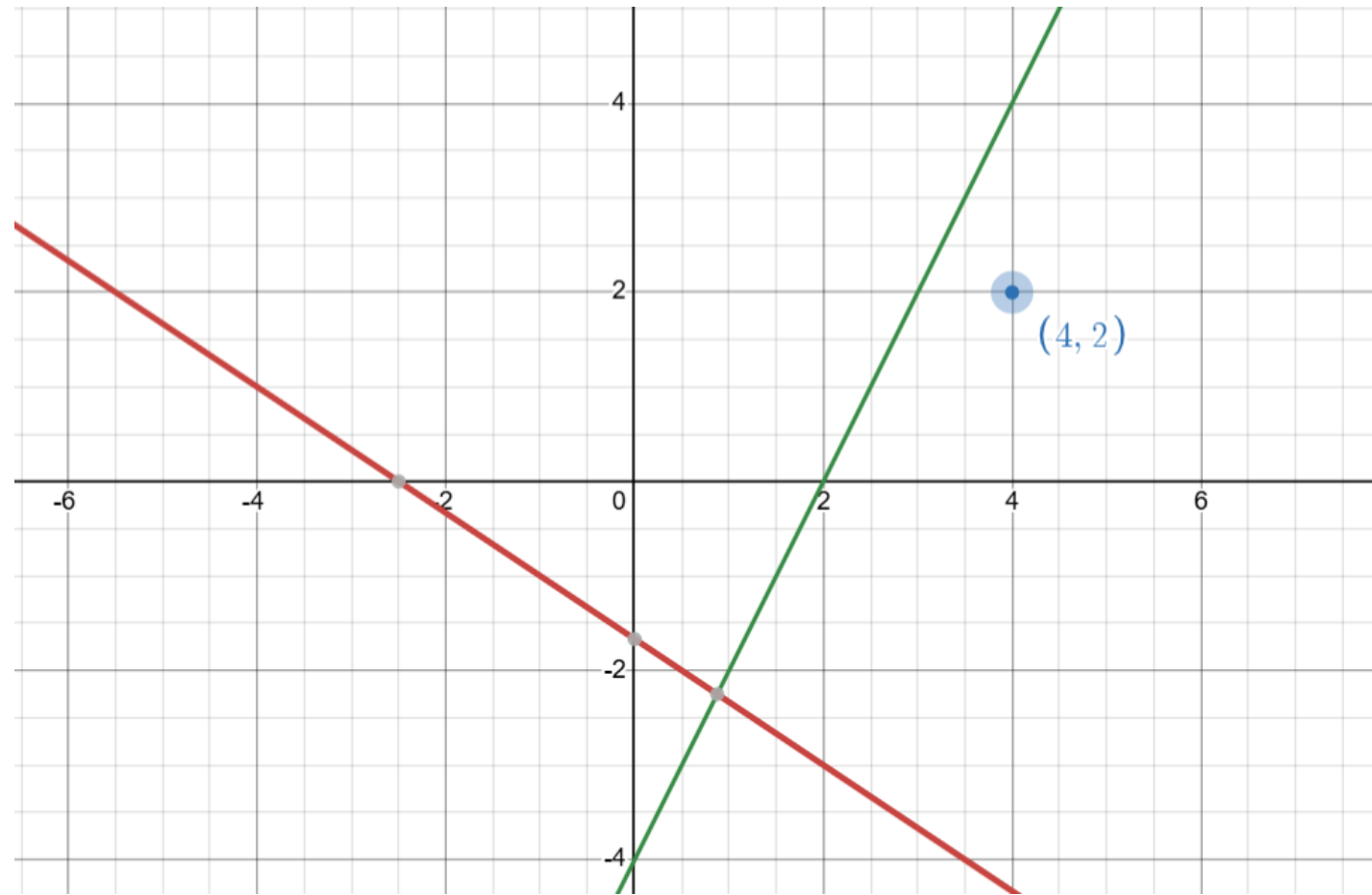
Perceptron Training – idea

- Decision boundary: $2x + 3y + 5 = 0$
- Let us say we have a point, P (4,2) which belongs to a negative class i.e. it was wrongly predicted as a positive class.
- We need to shift the decision boundary to the right of the point P.



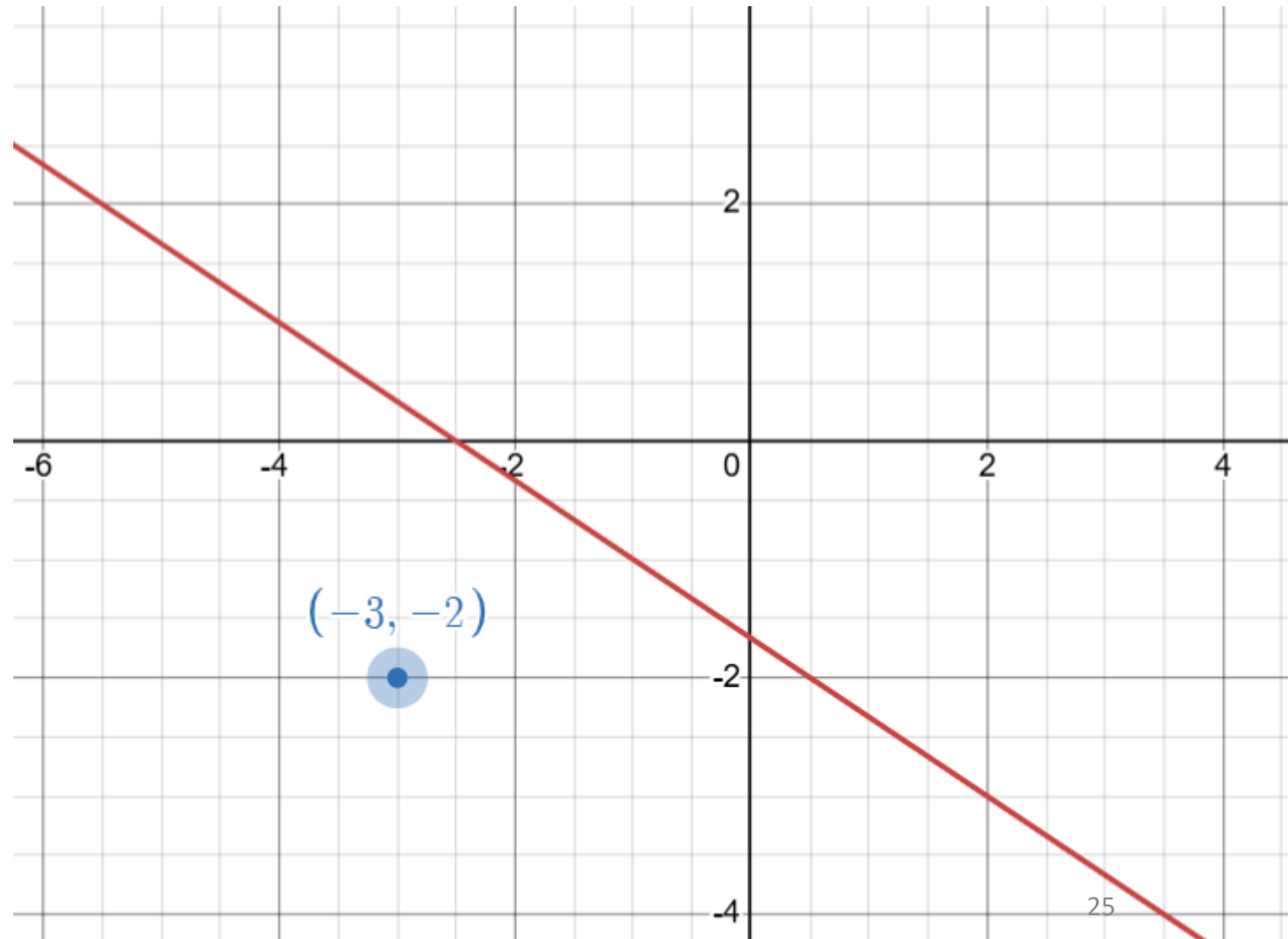
Perceptron Training – idea

	2	3	5
	4	2	1
(-)	-2	1	4



Perceptron Training – idea

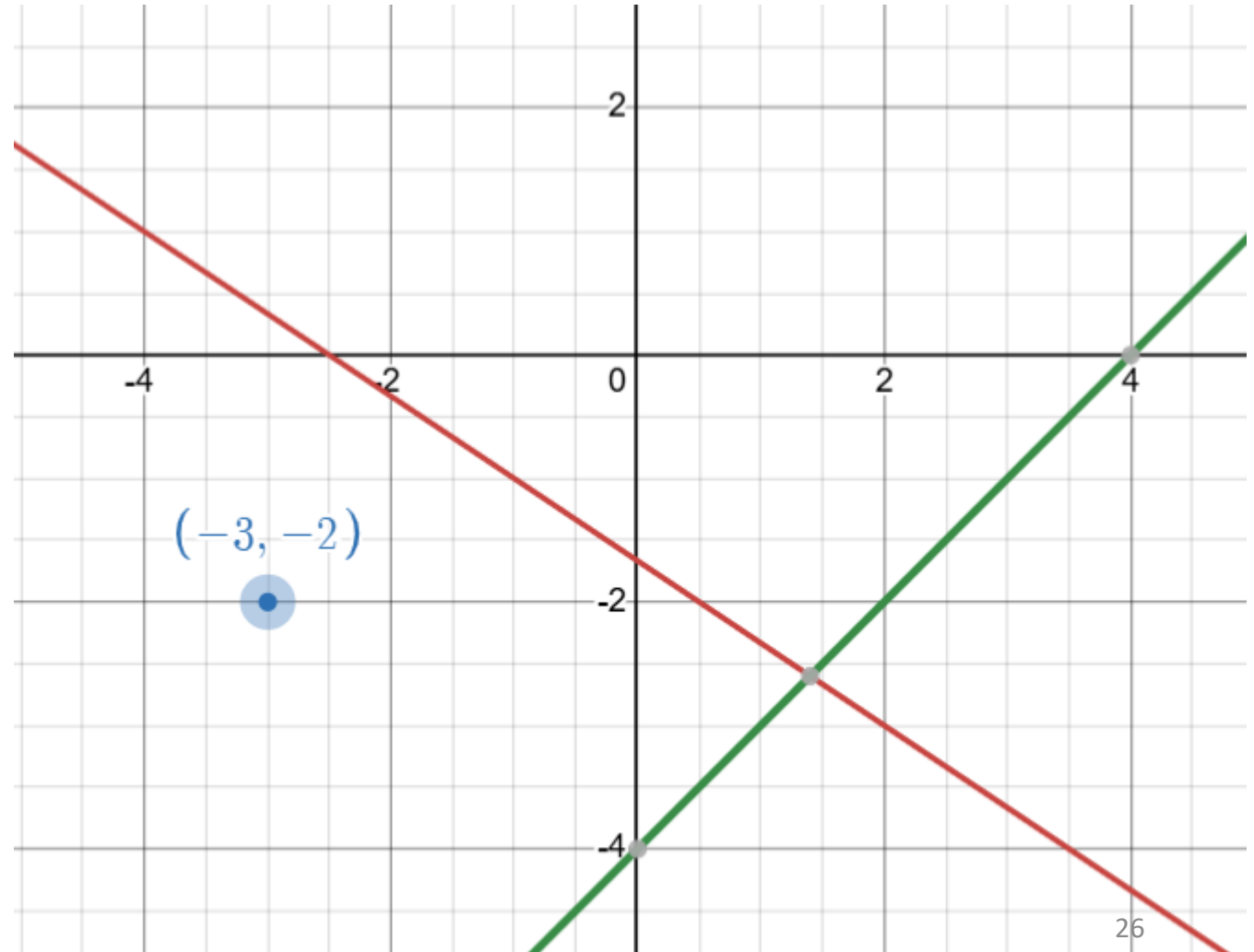
- Decision boundary: $2x + 3y + 5 = 0$
- Let us say we have a point, Q $(-3, -2)$ which belongs to a positive class i.e. it was wrongly predicted as a negative class.
- We need to shift the decision boundary to the right of the point P.



Perceptron Training – idea

- Decision boundary: $2x + 3y + 5 = 0$
- Let us say we have a point, Q (-3,-2) which belongs to a positive class i.e. it was wrongly predicted as a negative class.
- We need to shift the decision boundary to the right of the point P.

	2	3	5
	-3	-2	1
(+)	-1	1	4



Learning Algorithm

Algorithm 1: Perceptron Learning Algorithm

epochs = 1000

for i **in** **range**(**epochs**)

 Randomly select a point

if $x_i \in N$ **and** $\sum_{i=0}^n w_i x_i \geq 0$

$$w_{new} = w_{old} - x_i$$

if $x_i \in P$ **and** $\sum_{i=0}^n w_i x_i < 0$

$$w_{new} = w_{old} + x_i$$

- The updates of the bias and weights are too abrupt.
- In order to make the learning more gradual, we need to introduce a learning rate, η

Learning Algorithm

Algorithm 2: Perceptron Learning Algorithm

epochs = 1000

η = 0.01

for i **in** **range**(epochs)

 Randomly select a point

if $x_i \in N$ **and** $\sum_{i=0}^n w_i x_i \geq 0$

$$w_{new} = w_{old} - \eta x_i$$

if $x_i \in P$ **and** $\sum_{i=0}^n w_i x_i < 0$

$$w_{new} = w_{old} + \eta x_i$$

Learning Algorithm

Algorithm 3: Final Perceptron Learning Algorithm (epochs, η)

for i in range(epochs)

 Randomly select a point

$$w_{new} = w_{old} + \eta(y_i - \hat{y}_i)x_i$$

Problems with Perceptron rule / Perceptron trick

- Convergence can be slow
 - If the **learning rate** (η) is too small, updates take too long.
 - If η is too large, it may **oscillate** and never converge.
- Limited generalization ability
 - The perceptron only finds **one possible hyperplane**, but there are often many valid solutions.
 - It **does not optimize for the best margin**, meaning it may not generalize well to new data.

Loss Function

- A **loss function** is a mathematical function that measures how well a machine learning model's predictions match the actual values. It quantifies the error between predicted outputs and the true labels, guiding the model's learning process.

Why do we need a loss function?

- It **evaluates the model's performance** by computing the difference between predictions and actual values.
- It **guides optimization algorithms** (like gradient descent) to update the model's parameters (weights & biases).
- A **smaller loss means a better model**.

Examples of Loss Function

- Hinge loss – SVM
- Cross Entropy loss / Log loss – Logistic Regression
- MSE – Regression
- MAE – Regression
- Huber Loss – Regression (less sensitive to outliers compared to MSE)

Loss Function (sklearn documentation)

1.5.8. Mathematical formulation

We describe here the mathematical details of the SGD procedure. A good overview with convergence rates can be found in [\[12\]](#).

<https://scikit-learn.org/stable/modules/sgd.html#>

Given a set of training examples $(x_1, y_1), \dots, (x_n, y_n)$ where $x_i \in \mathbf{R}^m$ and $y_i \in \mathbf{R}$ ($y_i \in -1, 1$ for classification), our goal is to learn a linear scoring function $f(x) = w^T x + b$ with model parameters $w \in \mathbf{R}^m$ and intercept $b \in \mathbf{R}$. In order to make predictions for binary classification, we simply look at the sign of $f(x)$. To find the model parameters, we minimize the regularized training error given by

$$E(w, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \alpha R(w)$$

Loss Functions

Different choices for L entail different classifiers or regressors:

- Hinge (soft-margin): equivalent to Support Vector Classification.
 $L(y_i, f(x_i)) = \max(0, 1 - y_i f(x_i))$.
- Perceptron: $L(y_i, f(x_i)) = \max(0, -y_i f(x_i))$.
- Modified Huber: $L(y_i, f(x_i)) = \max(0, 1 - y_i f(x_i))^2$ if $y_i f(x_i) > -1$, and $L(y_i, f(x_i)) = -4y_i f(x_i)$ otherwise.
- Log Loss: equivalent to Logistic Regression.
 $L(y_i, f(x_i)) = \log(1 + \exp(-y_i f(x_i)))$.
- Squared Error: Linear regression (Ridge or Lasso depending on R).
 $L(y_i, f(x_i)) = \frac{1}{2} (y_i - f(x_i))^2$.
- Huber: less sensitive to outliers than least-squares. It is equivalent to least squares when $|y_i - f(x_i)| \leq \varepsilon$, and $L(y_i, f(x_i)) = \varepsilon |y_i - f(x_i)| - \frac{1}{2} \varepsilon^2$ otherwise.
- Epsilon-Insensitive: (soft-margin) equivalent to Support Vector Regression.
 $L(y_i, f(x_i)) = \max(0, |y_i - f(x_i)| - \varepsilon)$.

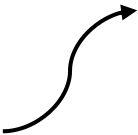
Perceptron Loss Function

$$L(y_i, f(x_i)) = \max(0, -y_i f(x_i))$$
$$L(w_1, w_2, b) = \frac{1}{n} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

Objective

$$L = \underset{w_1, w_2, b}{\operatorname{argmin}} \sum_{i=1}^n \max(0, -y_i f(x_i))$$

Gradient descent



Understanding Perceptron Loss Function

1	2	
x_1	x_2	y
x_{11}	x_{21}	-1/1
x_{21}	x_{22}	-1/1
...		
x_{i1}	x_{i2}	-1/1
...	...	...
x_{n1}	x_{n2}	-1/1

$$f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$$

For two points $\longrightarrow L = \frac{1}{2} [\max(0, -y_1 f(x_1)) + \max(0, -y_2 f(x_2))]$

Understanding Perceptron Loss Function

y_i	$\hat{y}_i$	$-y_i f(x_i)$	$\max(0, -y_i f(x_i))$
1	1	-1	0
-1	-1	-1	0
1	-1	1	1
-1	1	1	1